Computer Programming 143 – Lecture 11
Functions III

Faculty of Engineering

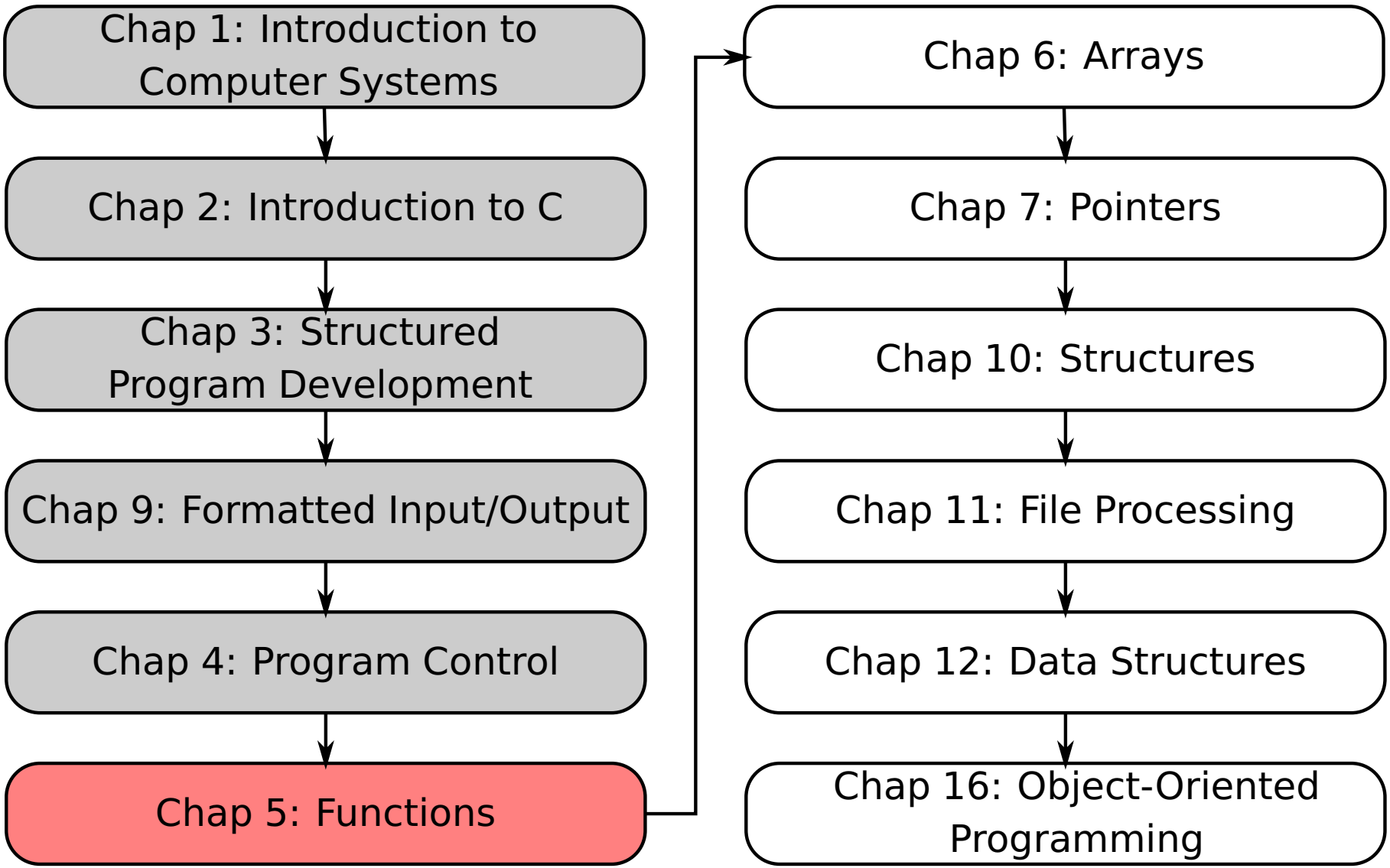
Copyright & Disclaimer

Disclaimer

This content is provided without warranty or representation of any kind. The use of the content is entirely at your own risk and Stellenbosch University (SU) will have no liability directly or indirectly as a result of this content.

The content must not be assumed to provide complete coverage of the particular study material. Content may be removed or changed without notice.

Module Overview



Lecture Overview

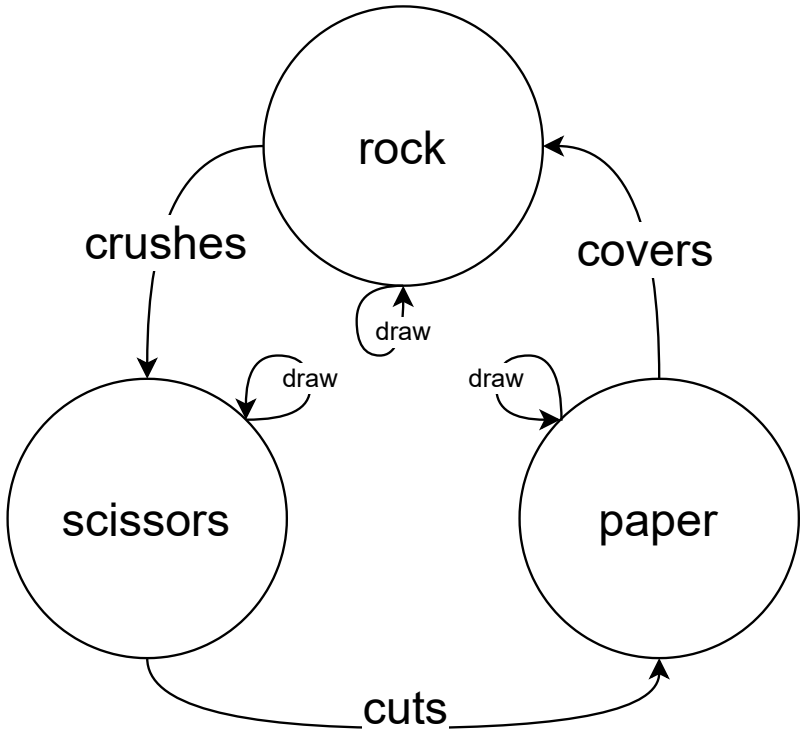
1. Example: A Game of Chance (5.11)
2. Storage Classes (5.12)
3. Scope Rules (5.13)

5.11 Example: A Game of Chance I

Rules of rock, paper, scissors

Each player simultaneously forms one of three shapes with an outstretched hand. The shapes are "rock", "paper", and "scissors". A simultaneous, zero-sum game, it has three possible outcomes: a draw, a win, or a loss. "rock" breaks (wins) "scissors", "scissors" cuts (wins) "paper", "paper" covers (wins) "rock". If both players choose the same shape, the game is a tie.

5.11 Example: A Game of Chance II



C code

Refer to Fig. 5.7 in Deitel & Deitel

5.11 Example: A Game of Chance III

C code

```c
// fig05_07.c
// Simulating the game of rock, paper, scissors
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h> // contains prototype for function time

enum GameStatus {CONTINUE, GAME_WON, GAME_LOST}; // constants represent g
// status
enum RoundStatus {DRAW, WON, LOST}; // constants represent round status
enum Shape {ROCK, PAPER, SCISSORS}; // constants representing possible sh
enum Shape computerPlayRandomShape(void); // playRandomShape function
// prototype;
enum RoundStatus getRoundStatus(enum Shape, enum Shape);
enum Shape convertIntToShape(int);
char* getStringFromShape(enum Shape);
```

5.12 Storage Classes I

Storage class specifiers

An identifier's storage class determines:

- **Storage duration** – how long a variable exists in memory
- **Scope** – where object can be referenced in program
- **Linkage** – specifies the files in which an identifier is known (more in Chapter 14)

5.12 Storage Classes II

Local variables

- Variables exist for certain section of program (block of code)
- Exist while block is active
- Destroyed when block is exited

Static storage

- Variables exist for entire program execution
- **`static`**: local variables defined in functions.
  - Keep value after function ends
  - Only known in their own function
- Global variables and functions
  - Known in any function

5.12 Storage Classes III

Example

```
void accumulate(void)
{
     static int acc = 0;  // static variable definition
     int num;  // local variable definition
     printf("Enter a number: ");
     scanf("%d", &num);
     acc += num;  // add number to current total
     printf("The current total is %d\n", acc);
}
```

5.13 Scope Rules I

Definition of scope

Portion of the program in which the identifier can be referenced

File scope

- Variable defined outside a function
- Known in all functions (in file)
- Used for global variables, function definitions, function prototypes

Block scope

- Variable declared inside a block (`{...}`)
- Used for variables, function parameters (local variables of function)
- Outer blocks “hidden” from inner blocks if there is a variable with the same name in the inner block

5.13 Scope Rules II

Example

```c
/* Example indicating scope of variable */
#include <stdio.h>

void useLocal(void);  // function prototype
void useStaticLocal(void);  // function prototype
void useGlobal(void);  // function prototype

int x = 1;  // global variable
```

5.13 Scope Rules III

Example (cont...)

```
int main(void)
{
    int x = 5;  // local variable to main

    printf("local x in scope of main is %d\n", x);
    useLocal();  // useLocal has local x
    useStaticLocal();  // useStaticLocal has static local
    useGlobal();  // useGlobal uses global x
    useLocal();  // useLocal has local x
    useStaticLocal();  // useStaticLocal has static local
    useGlobal();  // useGlobal uses global x
    printf("local x in scope of main is %d\n", x);

    return 0;
}
```

5.13 Scope Rules IV

Example (cont...)

```
// useLocal reinitializes local variable x during each call
void useLocal(void)
{
    int x = 25;  // initialized each time useLocal is called
    printf("\nlocal x in useLocal is %d ", x);
    printf("after entering useLocal\n");
    ++x;
    printf("local x in useLocal is %d ", x);
    printf("before exiting useLocal\n");
}
```

5.13 Scope Rules V

Example (cont...)

```
/*
useStaticLocal initializes static local variable x only
the first time the function is called;
value of x is saved between calls to this function
*/
void useStaticLocal(void)
{
    static int x = 50;  // initialized only once
    printf("\nlocal x in useStaticLocal is %d ", x);
    printf("after entering useStaticLocal\n");
    ++x;
    printf("local x in useStaticLocal is %d ", x);
    printf("before exiting useStaticLocal\n");
}
```

5.13 Scope Rules VI

Example (cont...)

```
// function useGlobal modifies global variable x during each call
void useGlobal(void)
{
    printf("\nglobal x is %d after entering useGlobal\n", x);
    x *= 10;
    printf("global x is %d before exiting useGlobal\n", x);
}
```

Use globals sparingly! Preferably never

5.13 Scope Rules VII

Output:

```
local x in scope of main is 5

local x in useLocal is 25 after entering useLocal
local x in useLocal is 26 before exiting useLocal

local x in useStaticLocal is 50 after entering useStaticLocal
local x in useStaticLocal is 51 before exiting useStaticLocal

global x is 1 after entering useGlobal
global x is 10 before exiting useGlobal
```

5.13 Scope Rules VIII

Output: (cont...)

```
local x in useLocal is 25 after entering useLocal
local x in useLocal is 26 before exiting useLocal

local x in useStaticLocal is 51 after entering useStaticLocal
local x in useStaticLocal is 52 before exiting useStaticLocal

global x is 10 after entering useGlobal
global x is 100 before exiting useGlobal

local x in scope of main is 5
```

Perspective

Today

Functions III

- Example: a game of chance
- Storage class
- Scope rules

Next lecture

Functions IV

- Recursion

Homework

1. Study Sections 5.11-5.13 in Deitel & Deitel
2. Do Self Review Exercise 5.6 in Deitel & Deitel
3. Do Exercise 5.27 in Deitel & Deitel